

The Entity-Relationship Model

This chapter defines all the major entity-relationship (ER) concepts that can be applied to the conceptual data modeling phase of the database life cycle. In Section 2.1, we will look at the simple level of ER modeling described in the original work by Chen and extended by others. The simple form of the ER model is used as the basis for effective communication with the end user about the conceptual database. Section 2.2 presents the more advanced concepts; although they are less generally accepted, they are useful to describe certain semantics that cannot be constructed with the simple model.

2.1 Fundamental ER Constructs

2.1.1 Basic Objects: Entities, Relationships, Attributes

The basic ER model consists of three classes of objects: entities, relationships, and attributes.

Entities

Entities are the principal data objects about which information is to be collected; they usually denote a person, place, thing, or event of informational interest. A particular occurrence of an entity is called an *entity instance* or sometimes an *entity occurrence*. In our example, employee,

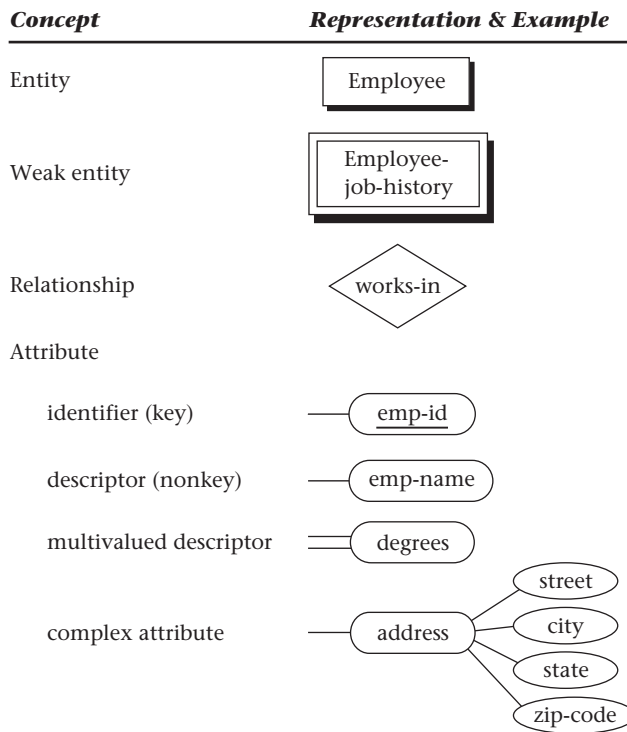


Figure 2.1 The basic ER model

department, division, project, skill, and location are all examples of entities. For easy reference, entity names will henceforth be capitalized throughout this text (e.g., Employee, Department, and so forth). The entity construct is a rectangle as depicted in Figure 2.1. The entity name is written inside the rectangle.

Relationships

Relationships represent real-world associations among one or more entities, and, as such, have no physical or conceptual existence other than that which depends upon their entity associations. Relationships are described in terms of degree, connectivity, and existence. These terms are defined in the sections that follow. The most common meaning associated with the term *relationship* is indicated by the connectivity between entity occurrences: one-to-one, one-to-many, and many-to-many. The relationship construct is a diamond that connects the associ-

ated entities, as shown in Figure 2.1. The relationship name can be written inside or just outside the diamond.

A *role* is the name of one end of a relationship when each end needs a distinct name for clarity of the relationship. In most of the examples given in Figure 2.2, role names are not required because the entity names combined with the relationship name clearly define the individual roles of each entity in the relationship. However, in some cases role names should be used to clarify ambiguities. For example, in the first case in Figure 2.2, the recursive binary relationship “manages” uses two roles, “manager” and “subordinate,” to associate the proper connectivities with the two different roles of the single entity. Role names are typically nouns. In this diagram, one employee’s role is to be the “manager” of up to n other employees. The other role is for particular “subordinates” to be managed by exactly one other employee.

Attributes

Attributes are characteristics of entities that provide descriptive detail about them. A particular occurrence of an attribute within an entity or relationship is called an attribute value. Attributes of an entity such as Employee may include emp-id, emp-name, emp-address, phone-no, fax-no, job-title, and so on. The attribute construct is an ellipse with the attribute name inside (or an oblong, as shown in Figure 2.1). The attribute is connected to the entity it characterizes.

There are two types of attributes: identifiers and descriptors. An identifier (or key) is used to uniquely determine an instance of an entity; a descriptor (or nonkey attribute) is used to specify a nonunique characteristic of a particular entity instance. Both identifiers and descriptors may consist of either a single attribute or some composite of attributes. For example, an identifier or key of Employee is emp-id, and a descriptor of Employee is emp-name or job-title. Key attributes are underlined in the ER diagram, as shown in Figure 2.1. We note, briefly, that you can have more than one identifier (key) for an entity, or you can have a set of attributes that compose a key (see Section 6.1.2).

Some attributes, such as specialty-area, may be multivalued. The notation for multivalued attributes is a double attachment line, as shown in Figure 2.1. Other attributes may be complex, such as an address that further subdivides into street, city, state, and zip code. Complex attributes are constructed to have attributes of their own; sometimes, however, the individual parts of a complex attribute are specified as individual attributes in the first place. Either form is reasonable in ER notation.

Entities have internal identifiers that uniquely determine the existence of entity instances, but weak entities derive their identity from the identifying attributes of one or more “parent” entities. Weak entities are often depicted with a double-bordered rectangle (see Figure 2.1), which denotes that all occurrences of that entity depend on an associated (strong) entity for their existence in the database. For example, in Figure 2.1, the weak entity *Employee-job-history* is related to the entity *Employee* and dependent upon *Employee* for its own existence.

2.1.2 Degree of a Relationship

The degree of a relationship is the number of entities associated in the relationship. Binary and ternary relationships are special cases where the degrees are 2 and 3, respectively. An n -ary relationship is the general form for any degree n . The notation for degree is illustrated in Figure 2.2. The binary relationship, an association between two entities, is by far the most common type in the natural world. In fact, many modeling systems use only this type. In Figure 2.2, we see many examples of the association of two entities in different ways: *Department* and *Division*, *Department* and *Employee*, *Employee* and *Project*, and so on. A binary recursive relationship (for example, “manages” in Figure 2.2) relates a particular *Employee* to another *Employee* by management. It is called recursive because the entity relates only to another instance of its own type. The binary recursive relationship construct is a diamond with both connections to the same entity.

A ternary relationship is an association among three entities. This type of relationship is required when binary relationships are not sufficient to accurately describe the semantics of the association. The ternary relationship construct is a single diamond connected to three entities, as shown in Figure 2.2. Sometimes a relationship is mistakenly modeled as ternary when it could be decomposed into two or three equivalent binary relationships. When this occurs, the ternary relationship should be eliminated to achieve both simplicity and semantic purity. Ternary relationships are discussed in greater detail in Section 2.2.3 and Chapter 6.

An entity may be involved in any number of relationships, and each relationship may be of any degree. Furthermore, two entities may have any number of binary relationships between them, and so on for any n entities (see n -ary relationships defined in Section 2.2.4).

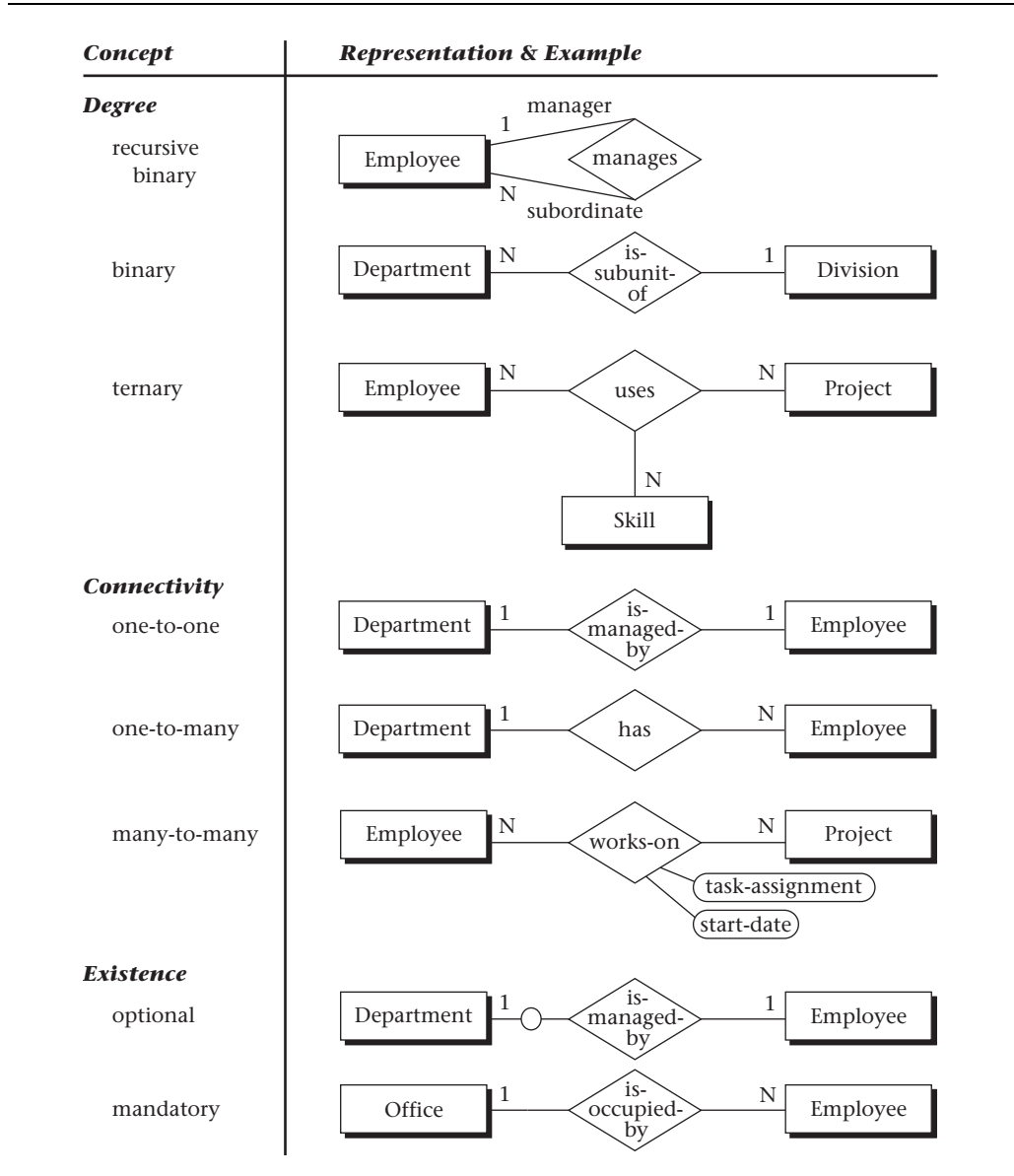


Figure 2.2 Degrees, connectivity, and attributes of a relationship

2.1.3 Connectivity of a Relationship

The *connectivity* of a relationship describes a constraint on the connection of the associated entity occurrences in the relationship. Values for connectivity are either “one” or “many.” For a relationship between the entities Department and Employee, a connectivity of one for Department and many for Employee means that there is at most one entity occurrence of Department associated with many occurrences of Employee. The actual count of elements associated with the connectivity is called the *cardinality* of the relationship connectivity; it is used much less frequently than the connectivity constraint because the actual values are usually variable across instances of relationships. Note that there are no standard terms for the connectivity concept, so the reader is admonished to consider the definition of these terms carefully when using a particular database design methodology.

Figure 2.2 shows the basic constructs for connectivity for binary relationships: one-to-one, one-to-many, and many-to-many. On the “one” side, the number one is shown on the connection between the relationship and one of the entities, and on the “many” side, the letter *N* is used on the connection between the relationship and the entity to designate the concept of many.

In the one-to-one case, the entity Department is managed by exactly one Employee, and each Employee manages exactly one Department. Therefore, the minimum and maximum connectivities on the “is-managed-by” relationship are exactly one for both Department and Employee.

In the one-to-many case, the entity Department is associated with (“has”) many Employees. The maximum connectivity is given on the Employee (many) side as the unknown value *N*, but the minimum connectivity is known as one. On the Department side the minimum and maximum connectivities are both one, that is, each Employee works within exactly one Department.

In the many-to-many case, a particular Employee may work on many Projects and each Project may have many Employees. We see that the maximum connectivity for Employee and Project is *N* in both directions, and the minimum connectivities are each defined (implied) as one.

Some situations, though rare, are such that the actual maximum connectivity is known. For example, a professional basketball team may be limited by conference rules to 12 players. In such a case, the number

12 could be placed next to an entity called “team members” on the many side of a relationship with an entity “team.” Most situations, however, have variable connectivity on the many side, as shown in all the examples of Figure 2.2.

2.1.4 Attributes of a Relationship

Attributes can be assigned to certain types of relationships as well as to entities. An attribute of a many-to-many relationship, such as the “works-on” relationship between the entities Employee and Project (Figure 2.2), could be “task-assignment” or “start-date.” In this case, a given task assignment or start date only has meaning when it is common to an instance of the assignment of a particular Employee to a particular Project via the relationship “works-on.”

Attributes of relationships are typically assigned only to binary many-to-many relationships and to ternary relationships. They are not normally assigned to one-to-one or one-to-many relationships, because of potential ambiguities. For example, in the one-to-one binary relationship “is-managed-by” between Department and Employee, an attribute “start-date” could be applied to Department to designate the start date for that department. Alternatively, it could be applied to Employee as an attribute for each Employee instance, to designate the employee’s start date as the manager of that department. If, instead, the relationship is many-to-many, so that an employee can manage many departments over time, then the attribute “start-date” must shift to the relationship, so each instance of the relationship that matches one employee with one department can have a unique start date for that employee as manager of that department.

2.1.5 Existence of an Entity in a Relationship

Existence of an entity occurrence in a relationship is defined as either mandatory or optional. If an occurrence of either the “one” or “many” side entity must always exist for the entity to be included in the relationship, then it is mandatory. When an occurrence of that entity need not always exist, it is considered optional. For example, in Figure 2.2 the entity Employee may or may not be the manager of any Department, thus making the entity Department in the “is-managed-by” relationship between Employee and Department optional.

Table 6.4 Candidate Tables (and FDs) from ER Diagram Transformation (*continued*)

<i>technician</i>	none
<i>skill</i>	skill_type -> skill_descrip
<i>project</i>	project_name -> start_date, end_date, head_id
<i>location</i>	loc_name -> loc_county, loc_state, zip
<i>prof_assoc</i>	assoc_name -> assoc_addr, phone_no, start_date
<i>desktop</i>	desktop_no -> computer_type, serial_no desktop_no -> emp_no
<i>assigned_to</i>	emp_id, loc_name -> project_name
<i>skill_used</i>	none

6.4 Determining the Minimum Set of 3NF Tables

A minimum set of 3NF tables can be obtained from a given set of FDs by using the well-known synthesis algorithm developed by Bernstein [1976]. This process is particularly useful when you are confronted with a list of hundreds or thousands of FDs that describe the semantics of a database. In practice, the ER modeling process automatically decomposes this problem into smaller subproblems: the attributes and FDs of interest are restricted to those attributes within an entity (and its equivalent table) and any foreign keys that might be imposed upon that table. Thus, the database designer will rarely have to deal with more than ten or twenty attributes at a time, and in fact most entities are initially defined in 3NF already. For those tables that are not yet in 3NF, only minor adjustments will be needed in most cases.

In the following text we briefly describe the synthesis algorithm for those situations where the ER model is not useful for the decomposition. In order to apply the algorithm, we make use of the well-known Armstrong axioms, which define the basic relationships among FDs.

Inference rules (Armstrong axioms)

<i>Reflexivity</i>	If Y is a subset of the attributes of X, then $X \rightarrow Y$ (i.e., if X is ABCD and Y is ABC, then $X \rightarrow Y$. Trivially, $X \rightarrow X$)
<i>Augmentation</i>	If $X \rightarrow Y$ and Z is a subset of table R (i.e., Z is any attribute in R), then $XZ \rightarrow YZ$.

Appendix

The Basics of SQL

Structured Query Language, or SQL, is the ISO-ANSI standard data definition and manipulation language for relational database management systems. Individual relational database systems use slightly different dialects of SQL syntax and naming rules, and these differences can be seen in the SQL user guides for those systems. In this text, as we explore each step of the logical and physical design portion of the database life cycle, many examples of database table creation and manipulation make use of SQL syntax.

Basic SQL use can be learned quickly and easily by reading this appendix. The more advanced features, such as statistical analysis and presentation of data, require additional study and are beyond the reach of the typical nonprogrammer. However, the DBA can create SQL views to help nonprogrammers set up repetitive queries; other languages, such as forms, are being commercially sold for nonprogrammers. For the advanced database programmer, embedded SQL (in C programs, for instance) is widely available for the most complex database applications, which need the power of procedural languages.

This appendix introduces the reader to the basic constructs for the SQL-99 (and SQL-92) database definition, queries, and updates through a sequence of examples with some explanatory text. We start with a definition of SQL terminology for data types and operators. This is followed by an explanation of the data definition language (DDL) constructs using the “create table” commands, including a definition of the various

types of integrity constraints, such as foreign keys and referential integrity. Finally, we take a detailed look at the SQL-99 data manipulation language (DML) features through a series of simple and then more complex practical examples of database queries and updates.

The specific features of SQL, as implemented by the major vendors IBM, Oracle, and Microsoft, can be found in the references at the end of this appendix.

A.1 SQL Names and Operators

This section gives the basic rules for SQL-99 (and SQL-92) data types and operators.

- *SQL names*: Although these have no particular restrictions, some vendor-specific versions of SQL do have some restrictions. For example, in Oracle, names of tables and columns (attributes) can be up to 30 characters long, must begin with a letter, and can include the symbols (a-z, 0-9,_,\$,#). Names should not duplicate reserved words or names for other objects (attributes, tables, views, indexes) in the database.
- *Data types for attributes*: character, character varying, numeric, decimal, integer, smallint, float, double precision, real, bit, bit varying, date, time, timestamp, interval.
- *Logical operators*: and, or, not, ().
- *Comparison operators*: =, <>, <, <=, >, >=, (), in, any, some, all, between, not between, is null, is not null, like.
- *Set operators*:
 - union: combines queries to display any row in each subquery
 - intersect: combines queries to display distinct rows common to all subqueries
 - except: combines queries to return all distinct rows returned by the first query, but not the second (this is “minus” or “difference” in some versions of SQL)
- *Set functions*: count, sum, min, max, avg.
- *Advanced value expressions*: CASE, CAST, row value constructors. CASE is similar to CASE expressions in programming languages, in which a select command needs to produce different results when there are different values of the search condition. The CAST

expression allows you to convert data of one type to a different type, subject to some restrictions. Row value constructors allow you to set up multiple column value comparisons with a much simpler expression than is normally required in SQL (see Melton and Simon [1993] for detailed examples).

A.2 Data Definition Language (DDL)

The basic definitions for SQL objects (tables and views) are:

- *create table*: defines a table and all its attributes
- *alter table*: add new columns, drop columns, or modifies existing columns in a table
- *drop table*: deletes an existing table
- *create view, drop view*: defines/deletes a database view (see Section A.3.4)

Some versions of SQL also have *create index/drop index*, which defines/deletes an index on a particular attribute or composite of several attributes in a particular table.

The following table creation examples are based on a simple database of three tables: **customer**, **item**, and **order**. (Note that we put table names in boldface throughout the book for readability.)

```
create table customer
  (cust_num      numeric,
   cust_name     char(20),
   address       varchar(256),
   credit_level   numeric,
   check (credit_level >= 1000),
   primary key (cust_num));
```

Note that the attribute `cust_num` could be defined as “numeric not null unique” instead of explicitly defined as the primary key, since they both have the same meaning. However, it would be redundant to have both forms in the same table definition. The check rule is an important integrity constraint that tells SQL to automatically test each insertion of `credit_level` value for something greater than or equal to 1000. If not, an error message should be displayed.

```

create table item
    (item_num      numeric,
     item_name     char(20),
     price         numeric,
     weight        numeric,
     primary key (item_num));

create table order
    (ord_num       char(15),
     cust_num      numeric not null,
     item_num      numeric not null,
     quantity      numeric,
     total_cost    numeric,
     primary key (ord_num),
     foreign key (cust_num) references customer
         on delete no action on update cascade,
     foreign key (item_num) references item
         on delete no action on update cascade);

```

SQL, while allowing the above format for primary key and foreign key, recommends a more detailed format, shown below, for table **order**:

```

constraint pk_constr primary key (ord_num),
constraint fk_constr1 foreign key
    (cust_num) references customer
    (cust_num) on delete no action on update cascade,
constraint fk_constr2 foreign key
    (item_num) references item (item_num)
    on delete no action on update cascade);

```

in which `pk_constr` is a primary key constraint name, and `fk_constr1` and `fk_constr2` are foreign key constraint names. The word “constraint” is a keyword, and the object in parentheses after the table name is the name of the primary key in that table referenced by the foreign key.

The following constraints are common for attributes defined in the SQL *create table* commands:

- *Not null*: a constraint that specifies that an attribute must have a nonnull value.
- *Unique*: a constraint that specifies that the attribute is a candidate key; that is, that it has a unique value for every row in the table. Every attribute that is a candidate key must also have the constraint not null. The constraint unique is also used as a clause to

designate composite candidate keys that are not the primary key. This is particularly useful when transforming ternary relationships to SQL.

- *Primary key*: the primary key is a set of one or more attributes that, when taken collectively, enables us to uniquely identify an entity or table. The set of attributes should not be reducible (see Section 6.1.2). The designation primary key for an attribute implies that the attribute must be not null and unique, but the SQL keywords NOT NULL and UNIQUE are redundant for any attribute that is part of a primary key, and need not be specified in the create table command.
- *Foreign key*: the referential integrity constraint specifies that a foreign key in a referencing table column must match an existing primary key in the referenced table. The references clause specifies the name of the referenced table. An attribute may be both a primary key and a foreign key, particularly in relationship tables formed from many-to-many binary relationships or from n -ary relationships.

Foreign key constraints are defined for deleting a row on the referenced table and for updating the primary key of the referenced table. The referential trigger actions for delete and update are similar:

- *on delete cascade*: the delete operation on the referenced table “cascades” to all matching foreign keys.
- *on delete set null*: foreign keys are set to null when they match the primary key of a deleted row in the referenced table. Each foreign key must be able to accept null values for this operation to apply.
- *on delete set default*: foreign keys are set to a default value when they match the primary key of the deleted row(s) in the reference table. Legal default values include a literal value, “user,” “system user,” or “no action.”
- *on update cascade*: the update operation on the primary key(s) in the referenced table “cascades” to all matching foreign keys.
- *on update set null*: foreign keys are set to null when they match the old primary key value of an updated row in the referenced table. Each foreign key must be able to accept null values for this operation to apply.
- *on update set default*: foreign keys are set to a default value when they match the primary key of an updated row in the reference

table. Legal default values include a literal value, “user,” “system user,” or “no action.”

The *cascade* option is generally applicable when either the mandatory existence constraint or the ID dependency constraint is specified in the ER diagram for the referenced table, and either *set null* or *set default* is applicable when optional existence is specified in the ER diagram for the referenced table (see Chapters 2 and 5).

Some systems, such as DB2, have an additional option on delete or update, called *restricted*. Delete restricted means that the referenced table rows are deleted only if there are no matching foreign key values in the referencing table. Similarly, *update restricted* means that the referenced table rows (primary keys) are updated only if there are no matching foreign key values in the referencing table.

Various column and table constraints can be specified as *deferrable* (the default is *not deferrable*), which means that the DBMS will defer checking this constraint until you commit the transaction. Often this is required for mutual constraint checking.

The following examples illustrate the *alter table* and *drop table* commands. The first *alter table* command modifies the *cust_name* data type from *char(20)* in the original definition to *varchar(256)*. The second and third *alter table* commands add and drop a column, respectively. The *add column* option specifies the data type of the new column.

```
alter table customer
modify (cust_name varchar(256));

alter table customer
add column cust_credit_limit numeric;

alter table customer
drop column credit_level;

drop table customer;
```

A.3 Data Manipulation Language (DML)

Data manipulation language commands are used for queries, updates, and the definition of views. These concepts are presented through a series of annotated examples, from simple to moderately complex.

A.3.1 SQL Select Command

The SQL select command is the basis for all database queries. The following series of examples illustrates the syntax and semantics of the select command for the most frequent types of queries in everyday business applications. To illustrate each of the commands, we assume the following set of data in the database tables:

customer table

<u>cust_num</u>	<u>cust_name</u>	<u>address</u>	<u>credit_level</u>
001	Kirk	Enterprise	10
002	Spock	Enterprise	9
003	Scotty	Enterprise	8
004	Bones	Enterprise	8
005	Gorn	PlanetoidArena	1
006	Khan	CetiAlphaFive	2
007	Uhura	Enterprise	7
008	Chekov	Enterprise	6
009	Sulu	Enterprise	6

item table

<u>item_num</u>	<u>item_name</u>	<u>price</u>	<u>weight</u>
125	phaser	350	2
137	beam	1500	250
143	shield	4500	3000
175	fusionMissile	2750	500
211	captainsLog	50	2
234	starShip	25000	15000
356	sensor	245	15
368	intercom	1200	75
399	medicalKit	75	3

order table

<u>ord_num</u>	<u>cust_num</u>	<u>item_num</u>	<u>quantity</u>	<u>total_cost</u>
10012	005	125	2	700
10023	006	175	20	55000
10042	003	137	3	4500
10058	001	211	1	50
10232	007	368	1	1200
10266	002	356	50	12250
10371	004	399	10	750
11070	009	143	1	4500
11593	008	125	2	700
11775	006	125	3	1050
12001	001	234	1	25000

Basic Commands

1. Display the entire **customer** table. The asterisk (*) denotes that all records from this table are to be read and displayed.

```
select *
from customer;
```

This results in a display of the complete **customer** table (as shown above).

2. Display customer name, customer number, and credit level for all customers on the Enterprise who have a credit level greater than 7. Order by ascending sequence of customer name (the order-by options are *asc*, *desc*). Note that the first selection condition is specified in the *where* clause and succeeding selection conditions are specified by *and* clauses. Character type data and other non-numeric data are placed inside single quotes, but numeric data is given without quotes. Note that useful column names can be created by using formatting commands (which are not shown here).

```
select cust_name, cust_num, credit_level
from customer
where address = 'Enterprise'
and credit_level > 7
order by cust_name asc;
```

customer name	customer number	credit level
Bones	004	8
Kirk	001	10
Scotty	003	8
Spock	002	9

3. Display all customer and order item information (all columns), but omit customers with a credit level greater than 6. In this query, the *from* clause shows the definition of abbreviations **c** and **o** for tables **customer** and **order**, respectively. The abbreviations can be used anywhere in the query to denote their respective table names. This example also illustrates a *join* between tables **customer** and **order**, using the common attribute name *cust_num* as shown in the *where* clause. The join finds matching *cust_num* values from the two tables and displays all the data

from the matching rows, except where the credit number is 7 or above, and ordered by customer number.

```
select c.*, o.*
from customer as c, order as o
where c.cust_num = o.cust_num
and c.credit_level < 7
order by cust_no asc;
```

cust. no.	cust. name	address	credit level	order no.	item no.	qty	total	cost
005	Gorn	PlanetoidArena	1	10012	125	2	700	
006	Khan	CetiAlphaFive	2	11775	125	3	1050	
006	Khan	CetiAlphaFive	2	10023	175	20	55000	
008	Chekov	Enterprise	6	11593	125	2	700	
009	Sulu	Enterprise	6	11070	143	1	4500	

Union and Intersection Commands

1. Which items were ordered by customer 002 or customer 007? This query can be answered in two ways, one with a set operator (union) and the other with a logical operator (or).

```
select item_num, item_name, cust_num, cust_name
from order
where cust_num = 002
union
select item_num, item_name, cust_num, cust_name
from order
where cust_num = 007;
```

```
select item_num, item_name, cust_num, cust_name
from order
where (cust_num = 002 or cust_num = 007);
```

item number	item name	customer no.	customer name
356	sensor	002	Spock
368	intercom	007	Uhura

2. Which items are ordered by both customers 005 and 006? All the rows in table **order** that have customer 005 are selected and compared to the rows in **order** that have customer 006. Rows from each set are compared with all rows from the other set, and those

that have matching item numbers have the item numbers displayed.

```
select item_num, item_name, cust_num, cust_name
from order
where cust_num = 005
intersect
select item_num, item_name, cust_num, cust_name
from order
where cust_num = 006;
```

item number	item name	customer no	customer name
-----	-----	-----	-----
125	phaser	005	Gorn
125	phaser	006	Khan

Aggregate Functions

- 1. Display the total number of orders. This query uses the SQL function *count* to count the number of rows in table **order**.

```
select count(*)
from order;

count(order)
-----
11
```

- 2. Display the total number of customers actually placing orders for items. This is a variation of the count function and specifies that only the distinct number of customers is to be counted. The *distinct* modifier is required because duplicate values of customer numbers are likely to be found, since a customer can order many items and will appear in many rows of table order.

```
select count (distinct cust_num)
from order;

distinct count(order)
-----
9
```

3. Display the maximum quantity of an order of item number 125. The SQL *maximum* function is used to search the table **order**, select rows where the item number is 125, and display the maximum value of quantity from the rows selected.

```
select max (quantity)
from order
where item_num = 125;
```

```
max(quantity)
-----
3
```

4. For each type of item ordered, display the item number and total order quantity. Note that item_num and item_name in the select line must be in a *group by* clause. In SQL, any attribute to be displayed in the result of the select command must be included in a *group by* clause when the result of an SQL function is also to be displayed. The *group by* clause results in a display of the aggregate sum of quantity values for each value of item_num and item_name. The aggregate sums will be taken over all rows with the same value of item_num.

```
select item_num, item_name, sum(quantity)
from order
group by item_num, item_name;
```

item number	item name	sum(quantity)
125	phaser	7
137	beam	3
143	shield	1
175	fusionMissile	20
211	captainsLog	1
234	starShip	1
356	sensor	50
368	intercom	1
399	medicalKit	10

5. Display item numbers for all items ordered more than once. This query requires the use of the *group by* and *having* clauses to display data that is based on a count of rows from table **order** having the same value for attribute item_num.

```
select item_num, item_name
from order
group by item_num, item_name
having count(*) >1;
```

item number	item name
125	phaser

Joins and Subqueries

1. Display customer names of the customers who order item number 125. This query requires a *join* (*equijoin*) of tables **customer** and **order** to match customer names with item number 125. Including the item_num column in the output verifies that you have selected the item number you want. Note that the default ordering of output is typically ascending by the first column.

```
select c.cust_name, o.item_num
from customer as c, order as o
where c.cust_num = o.cust_num
and item_num = 125;
```

customer name	item number
Chekov	125
Gorn	125
Khan	125

This query can be equivalently performed with a *subquery* (sometimes called a nested subquery) with the following format. The *select* command inside the parentheses is a nested subquery and is executed first, resulting in a set of values for customer number (cust_num) selected from the **order** table. Each of those values is compared with cust_num values from the **customer** table, and matching values result in the display of customer name from the matching row in the **customer** table. This is effectively a join between tables **customer** and **order** with the selection condition of item number 125.

```

select cust_name, order_num
from customer
where cust_num in
      (select cust_num
       from order
       where item_num = 125);

```

2. Display names of customers who order at least one item priced over 1000. This query requires a three-level nested subquery format. Note that the phrases *in*, *= some*, and *= any* in the *where* clauses are often used as equivalent comparison operators; see Melton and Simon [1993].

```

select c.cust_name
from customer as c
where c.cust_num in
      (select o.cust_num
       from order as o
       where o.item_num = any
            (select i.item_num
             from item as i
             where i.price > 1000));

```

```

customer name
-----
Khan
Kirk
Scotty
Sulu
Uhura

```

3. Which customers have not ordered any item priced over 100? Note that one can equivalently use *not in* instead of *not any*. The query first selects the customer numbers from all rows from the join of tables **order** and **item** where the item price is over 100. Then it selects rows from table **customer** where the customer number does not match any of the customers selected in the subquery, and displays the customer names.

```

select c.cust_name
from customer as c
where c.cust_num not any

```

```
(select o.cust_num
from order as o, item as i
where o.item_num = i.item_num
and i.price > 100);
```

```
customer name
```

```
-----
```

```
Bones
```

4. Which customers have only ordered items weighing more than 1000? This is an example of the universal quantifier *all*. First the subquery selects all rows from table **item** where the item weight is over 1000. Then it selects rows from table **order** where all rows with a given item number match at least one row in the set selected in the subquery. Any rows in **order** satisfying this condition are joined with the **customer** table, and the customer name is displayed as the final result.

```
select c.cust_name
from customer as c, order as o
where c.cust_num = o.cust_num
and o.item_num = all
    (select i.item_num
     from item as i
     where i.weight > 1000);
```

```
customer name
```

```
-----
```

```
Sulu
```

Note that Kirk has ordered one item weighing over 1000 (star-Ship), but he has also ordered an item weighing under 1000 (captainsLog), so his name does not get displayed.

A.3.2 SQL Update Commands

The following SQL update commands relate to our continuing example and illustrate typical usage of insertion, deletion, and update of selected rows in tables.

This command adds one more customer (klinton) to the **customer** table:

```
insert into customer
values (010, 'klinton', 'rogueShip', 4);
```

This command deletes all customers with credit levels less than 2:

```
delete from customer
where credit_level < 2;
```

This command modifies the credit level of any customer with level 6 to level 7:

```
update customer
set credit_level = 7
where credit_level = 6;
```

A.3.3 Referential Integrity

The following update to the **item** table resets the value of `item_num` for a particular item, but because `item_num` is a foreign key in the **order** table, SQL must maintain referential integrity by triggering the execution sequence named by the foreign key constraint *on update cascade* in the definition of the **order** table (see Section A.2). This means that, in addition to updating a row in the **item** table, SQL will search the **order** table for values of `item_num` equal to 368 and reset each `item_num` value to 370.

```
update item
set item_num = 370
where item_num = 368;
```

If this update had been a *delete* instead, such as the following:

```
delete from item
where item_num = 368;
```

then the referential integrity trigger would have caused the additional execution of the foreign key constraint *on delete set default* in order (as defined in Section A.2), which finds every row in **order** with `item_num` equal to 368 and takes the action set up in the default. A typical action for this type of database might be to set `item_num` to either null or a predefined literal value to denote that the particular item has been

deleted; this would then be a signal to the system that the customer needs to be contacted to change the order. Of course, the system would have to be set up in advance to check for these values periodically.

A.3.4 SQL Views

A view in SQL is a named, derived (virtual) table that derives its data from base tables, the actual tables defined by the *create table* command. View definitions can be stored in the database, but the views (derived tables) themselves are not stored; they are derived at execution time when the view is invoked as a query using the SQL *select* command. The person who queries the view treats the view as if it were an actual (stored) table, unaware of the difference between the view and the base table.

Views are useful in several ways. First, they allow complex queries to be set up in advance in a view, and the novice SQL user is only required to make a simple query on the view. This simple query invokes the more complex query defined by the view. Thus, nonprogrammers are allowed to use the full power of SQL without having to create complex queries. Second, views provide greater security for a database, because the DBA can assign different views of the data to different users and control what any individual user sees in the database. Third, views provide a greater sense of data independence; that is, even though the base tables may be altered by adding, deleting, or modifying columns, the view query may not need to be changed. While view definition may need to be changed, that is the job of the DBA, not the person querying the view.

Views may be defined hierarchically; that is, a view definition may contain another view name as well as base table names. This enables some views to become quite complex.

In the following example, we create a view called “orders” that shows which items have been ordered by each customer and how many. The first line of the view definition specifies the view name and (in parentheses) lists the attributes of that view. The view attributes must correlate exactly with the attributes defined in the select statement in the second line of the view definition:

```
create view orders (customer_name, item_name,  
    quantity) as  
    select c.cust_name, i.item_name, o.quantity  
    from customer as c, item as i, order as o  
    where c.cust_num = o.cust_num  
    and o.item_num = i.item_num;
```

The *create view* command creates the view definition, which defines two joins among three base tables **customer**, **item**, and **order**; and SQL stores the definition to be executed later when invoked by a query. The following query selects all the data from the view “orders.” This query causes SQL to execute the select command given in the preceding view definition, producing a tabular result with the column headings for customer_name, item_name, and quantity.

```
select *
from orders;
```

Usually, views are not allowed to be updated, because the updates would have to be made to the base tables that make up the definition of the view. When a view is created from a single table, the view update is usually unambiguous, but when a view is created from the joins of multiple tables, the base table updates are very often ambiguous and may have undesirable side effects. Each relational system has its own rules about when views can and cannot be updated.

A.4 References

- Bulger B., Greenspan, J., and Wall, D. *MySQL/PHP Database Applications*, 2nd ed., Wiley, 2004.
- Gennick, J. *Oracle SQL*Plus: The Definitive Guide*, O'Reilly, 1999.
- Gennick, J. *SQL Pocket Guide*, O'Reilly, 2004.
- Melton, J., and Simon, A. R. *Understanding The New SQL: A Complete Guide*, Morgan Kaufmann, 1993.
- Mullins, C. S. *DB2 Developer's Guide*, 5th ed., Sams Publishing, 2004.
- Neilson, P. *Microsoft SQL Server Bible*, Wiley, 2003.
- van der Lans, R. *Introduction to SQL: Mastering the Relational Database Language*, 3rd ed., Addison-Wesley, 2000.

Exercises

ER and UML Conceptual Data Modeling

Problem 2-1

Draw a detailed ER diagram for an car rental agency database (e.g., Hertz), keeping track of the current rental location of each car, its current condition and history of repairs, customer information for a local office, expected return date, return location, car status (ready, being-repaired, currently-rented, being-cleaned). Select attributes from your intuition about the situation and list them separately from the diagram, but associated with a particular entity or relationship in the ER model.

Problem 2-2

Given the following assertions for a relational database that represents the current term enrollment at a large university, draw an ER diagram for this schema that takes into account all the assertions given. There are 2,000 instructors, 4,000 courses, and 30,000 students. Use as many ER constructs as you can to represent the true semantics of the problem.

Assertions:

- An instructor may teach one or more courses in a given term (average is 2.0 courses).
- An instructor must direct the research of at least one student (average = 2.5 students).
- A course may have none, one, or two prerequisites (average = 1.5 prerequisites).
- A course may exist even if no students are currently enrolled.
- Each course is taught by exactly one instructor.
- The average enrollment in a course is 30 students.
- A student must select at least one course per term (average = 4.0 course selections).

Problem 3-1

Draw UML class diagrams for an car rental agency database (e.g., Hertz), keeping track of the current rental location of each car, its current condition and history of repairs, customer information for a local office, expected return date, return location, car status (ready, being-repaired, currently-rented, being-cleaned). Select attributes from your intuition about the situation.

Draw one diagram showing the relationships of the classes without the attributes listed, then show each class individually with the attributes listed.

Problem 3-2

Given the following assertions for a relational database that represents the current term enrollment at a large university, draw an UML diagram for this schema that takes into account all the assertions given. There are 2,000 instructors, 4,000 courses, and 30,000 students. Use as many UML constructs as you can to represent the true semantics of the problem.

Assertions:

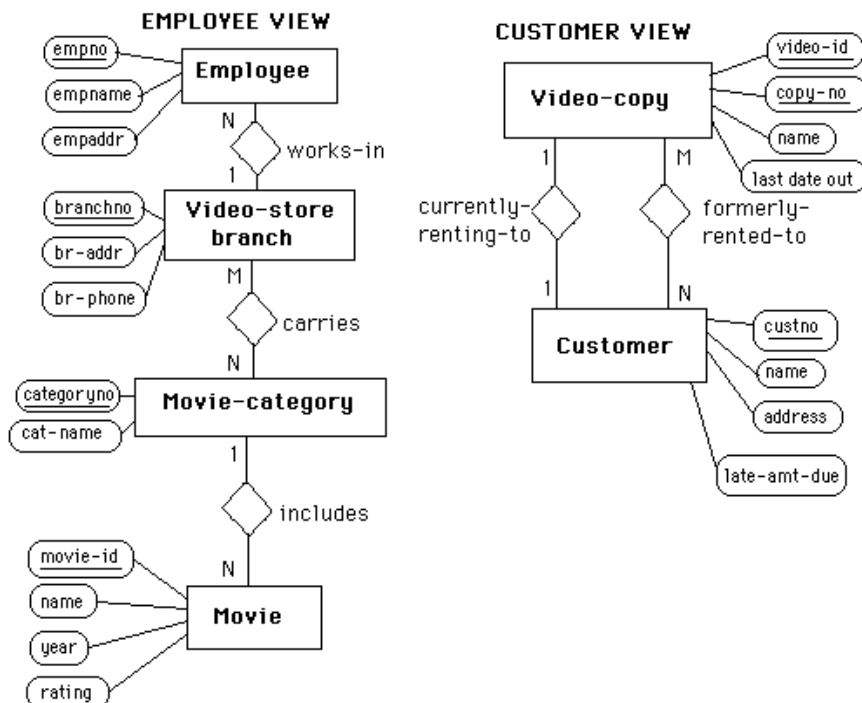
- An instructor may teach none, one, or more courses in a given term (average is 2.0 courses).

- An instructor must direct the research of at least one student (average = 2.5 students).
- A course may have none, one, or two prerequisites (average = 1.5 prerequisites).
- A course may exist even if no students are currently enrolled.
- Each course is taught by exactly one instructor.
- The average enrollment in a course is 30 students.
- A student must select at least one course per term (average = 4.0 course selections).

Conceptual Data Modeling and Integration

Problem 4-1

The following ER diagrams represent two views of a video store database as described to a database designer. Show how the two views can be integrated in the simplest and most useful way by making all necessary changes on the two diagrams. State any assumptions you need to make.



Transformation of the Conceptual Model to SQL

Problem 5-1

1. Transform your integrated ER diagram from Problem 4-1 into a SQL database with five to ten rows per table of data you make up to fit the database schema.
2. Demonstrate your database by displaying all the queries below:
 - a. Which video store branches have *Shrek* in stock (available) now?
 - b. In what section of the store (film category) can you find *Terminator*?
 - c. For customer "Anika Sorenstam," what titles are currently being rented and what are the overdue charges, if any?
 - d. Any query of your choice. (Show me what your system can really do!)

Normalization and Minimum Set of Tables

Problem 6-1

Given the table $R1(A, B, C)$ with FDs $A \rightarrow B$ and $B \rightarrow C$:

1. Is A a superkey for this table? _____
2. Is B a superkey for this table? _____
3. Is this table in 3NF, BCNF, or neither? _____

Problem 6-2

Given the table $R(A,B,C,D)$ with FDs $AB \rightarrow C$ and $BD \rightarrow A$:

1. What are all the superkeys of this table? _____
2. What are all the candidate keys for this table? _____
3. Is this table in 3NF, BCNF, or neither? _____

Problem 6-3

1. From the ER diagram given below and the resulting tables and candidate key attributes defined, what are all the functional dependencies (FDs) you can derive just by looking at the diagram and list of candidate keys?

<u>Table</u>	<u>Candidate Key(s)</u>
customer	cid
order	orderno
department	deptno
salesperson	sid
item	itemno
order-dept-sales	orderno,sid AND orderno,deptno
order-item-sales	orderno, itemno

<u>Table</u>	<u>FDs</u>
customer	
order	
department	
salesperson	
item	
order-dept-sales	
order-item-sales	

2. What is the level of normalization for each of these tables, based on the information given?

Problem 6-4

The following functional dependencies (FDs) represent a set of airline reservation system database constraints. Design a minimum set of BCNF tables, preserving all FDs, and express your solution in terms of the code letters given below (a time-saving device for your analysis). Is the set of tables you derived also BCNF?

reservation_no -> agent_no, agent_name, airline_name, flight_no,
passenger_name
reservation_no -> aircraft_type, departure_date, arrival_date,
departure_time, arrival_time

reservation_no -> departure_city, arrival_city, type_of_payment,
 seating_class, seat_no
 airline_name, flight_no -> aircraft_type, departure_time, arrival_time
 airline_name, flight_no -> departure_city, arrival_city, meal_type
 airline_name, flight_no, aircraft_type -> meal_type
 passenger_name -> home_address, home_phone, company_name
 aircraft_type, seat_no -> seating_class
 company_name -> company_address, company_phone
 company_phone -> company_name

A: reservation_no	L: departure_city
B: agent_no	M: arrival_city
C: agent_name	N: type_of_payment
D: airline_name	P: seating_class
E: flight_no	Q: seat_no
F: passenger_name	R: meal_type
G: aircraft_type	S: home_address
H: departure_date	T: home_phone
I: arrival_date	U: company_name
J: departure_time	V: company_address
K: arrival_time	W: company_phone

Problem 6-5

Given the following set of FDs, find the minimum set of 3NF tables. Designate the candidate key attributes of these tables. Is the set of tables you derived also BCNF?

1. J -> KLMNP
2. JKL -> MNP
3. K -> MQ
4. KL -> MNP
5. KM -> NP
6. N -> KP

Problem 6-6

Using only the given set of functional dependencies (FDs), find the minimum set of BCNF tables. Show the results of each step of Bernstein's algorithm separately. What are the candidate keys for each table?

1. $ABC \rightarrow DEFG$
2. $B \rightarrow CH$
3. $A \rightarrow HK$
4. $CD \rightarrow GKM$
5. $D \rightarrow CP$
6. $E \rightarrow FG$
7. $G \rightarrow CDM$

Problem 6-7

Given the FDs listed below, determine the minimum set of 3NF tables, preserving all FDs. Define the candidate keys of each table and determine which tables are also BCNF.

1. $ABC \rightarrow H$
2. $AC \rightarrow BDEFG$
3. $ACDF \rightarrow BG$
4. $AW \rightarrow BG$
5. $B \rightarrow ACF$
6. $H \rightarrow AXY$
7. $M \rightarrow NZ$
8. $MN \rightarrow HPT$
9. $XY \rightarrow MNP$

Problem 6-8

Given the following FDs, determine the minimum set of 3NF tables. Make sure that all FDs are preserved. Specify the candidate keys of each table. Note that each letter represents a separate data element (attribute). Is the set of tables you derived also BCNF?

- | | |
|-------------------------------|--------------------------|
| 1. $ABCD \rightarrow EFGHIJK$ | 8. $JK \rightarrow B$ |
| 2. $ACD \rightarrow JKLMN$ | 9. $MN \rightarrow ACD$ |
| 3. $A \rightarrow BH$ | 10. $L \rightarrow JK$ |
| 4. $B \rightarrow JKL$ | 11. $PQ \rightarrow S$ |
| 5. $BH \rightarrow PQR$ | 12. $PS \rightarrow JKQ$ |
| 6. $BL \rightarrow PS$ | 13. $PSR \rightarrow QT$ |
| 7. $EF \rightarrow ABCDH$ | |

Problem 6-9

Given the following FDs, determine the minimum set of 3NF tables. Make sure that all FDs are preserved. Specify the candidate keys of each table. Note that each letter represents a separate data element (attribute). Is the set of tables you derived also BCNF?

- | | |
|-------------------------|------------------------|
| 1. $A \rightarrow BGHJ$ | 5. $EB \rightarrow AF$ |
| 2. $AG \rightarrow HK$ | 6. $EF \rightarrow A$ |
| 3. $B \rightarrow K$ | 7. $H \rightarrow J$ |
| 4. $EA \rightarrow F$ | 8. $J \rightarrow AB$ |

Problem 6-10***FDs and MVDs***

Answer each question yes or no, and justify each answer. In most cases, you will be given a table R with a list of attributes, with at most one candidate key (the candidate key may be either a single attribute or composite attribute key, shown underlined).

Given table $R(\underline{A}, B, C, D)$ and the functional dependency $AB \rightarrow C$:

1. Is R in 3NF?
2. Is R in BCNF?
3. Does the multivalued dependency $AB \twoheadrightarrow C$ hold?
4. Does the set $\{R_1(A, B, C), R_2(A, B, D)\}$ satisfy the lossless join property?